Guide Grand Oral NSI - Échecs : script, code Python commenté et questions jury

Ce **guide** rassemble **tout** ce qu'un élève de **terminale** spécialité **NSI** doit avoir en main pour réussir son **Grand Oral du bac** avec les échecs comme **sujet** : méthodologie de l'**épreuve**, **choix** de problématique parmi trois angles, script minuté de **10 minutes**, code Python commenté, **questions** de **jury** rédigées, anti-sèche et **conseils** de **posture** pour le **jour** J.

L'objectif : que tu n'aies plus à chercher d'**exemples**, plus à compiler des sources d'**internet** dispersées, plus à inventer ton plan dans l'urgence. **Tout** est là, exploitable directement.

> ***Comment utiliser ce guide ?*** *Télécharge le PDF via le bouton en haut de la page pour l'avoir hors-ligne. Lis-le sur écran ou impression selon ce qui te convient. Entraîne-toi à expliquer le code à voix haute ligne par ligne : c'est l'exercice clé pour l'**oral NSI**.*

Comprendre l'épreuve : Grand Oral en spécialité NSI

Coefficient, durée et déroulé minute par minute

Le Grand Oral demande une présentation argumentée et un échange ; vérifie le coefficient applicable à ta session dans les informations officielles. Sa durée totale est de **40 minutes** :

Temps	Phase	Ce que tu fais	Ce que le jury observe
20 min	Préparation	Tu présentes deux **questions** préparées avec tes enseignants ; le **jury** en choisit une. Brouillon autorisé.	Rien : tu es seul en salle
10 min	Exposé debout	Tu présentes ta **question** debout, avec le support préparé pendant les 20 minutes de préparation.	**Posture**, voix, structure
10 min	Échange avec le **jury**	Le **jury** te questionne sur le **sujet**, ton plan, ton code.	Maîtrise, réactivité, ouverture

Attention aux guides encore en ligne qui décrivent un exposé de 5 minutes suivi d'une troisième partie de 5 minutes sur le projet d'orientation : c'était le format jusqu'à la session 2023. Depuis la session 2024, l'épreuve tient en deux temps de 10 minutes et la partie orientation a été supprimée.

Les 10 minutes d’exposé et les 10 minutes d’échange sont évaluées ensemble. Le coefficient n’attribue pas un point par minute : travaille aussi ta capacité à répondre aux questions.

Composition du jury et attendus

Le **jury** est composé de **deux** professeurs :

- L'un est issu de ta spécialité (typiquement **NSI** ou **mathématiques**)

- L’autre peut enseigner toute discipline, y compris la documentation ; il n’est pas nécessairement non-spécialiste

Ne suppose pas que le second est expert en **informatique**. Ton **oral** doit être compréhensible par un non-spécialiste : le code Python ne suffit pas, il faut le raconter en français clair. C'est le piège dans lequel tombent les candidats trop techniques.

Le ministère indique cinq dimensions : qualité orale, prise de parole en continu, connaissances, interaction avec le jury et construction de l’argumentation. Travaille chacune ; le projet d’orientation n’est pas une troisième partie chronométrée.

Trois pièges à éviter absolument

- **Réciter par cœur** : le **jury** entend la mémorisation. Préfère un canevas que tu adaptes au moment.
- **Lire tout son support** : cela peut gêner la prise de parole. Les notes préparées pendant les vingt minutes sont autorisées ; utilise-les comme appui.
- **Dépasser le temps** : à 10 minutes pile, le **jury** coupe. Mieux vaut conclure 30 secondes en avance que d'être interrompu.

Choisir ta problématique

La problématique est ta colonne vertébrale. En **NSI**, elle doit montrer que tu maîtrises une notion ancrée dans le **programme** de **terminale** : récursivité, coût d'un calcul ou arbres ; l'intelligence artificielle peut servir d'ouverture à valider avec ton enseignant.

Les trois angles recommandés

Angle A, Arbre de décision *(niveau accessible)*

> *« Comment un programme informatique peut-il prendre des décisions pour jouer aux échecs ? »*

Cet angle te fait raconter l'arbre des coups, la fonction d'évaluation, le **fonctionnement** de la récursivité. Bon **choix** si tu veux rester sur du solide et bien maîtrisé.

Angle B, Optimisation algorithmique *(niveau intermédiaire)*

> *« En quoi l'élagage alpha-bêta illustre-t-il l'optimisation d'une recherche arborescente ? »*

Plus technique, cet angle compare un coût en O(b^d) au meilleur cas de tri en O(b^(d/2)). C'est le **choix** privilégié quand tu maîtrises bien les **mathématiques** et que tu veux marquer des points sur le raisonnement.

Angle C, Intelligence artificielle et rupture épistémologique *(niveau avancé)*

> *« Pourquoi AlphaZero représente-t-il une rupture dans l'histoire de l'intelligence artificielle appliquée aux jeux de stratégie ? »*

Cet angle te place dans une réflexion plus large : opposition IA symbolique / IA connexionniste, **fonctionnement** des réseaux de neurones, **limites** de l'apprentissage automatique. À choisir si tu vises une école d'ingénieur ou une **mathématiques** spé.

Comment choisir ?

- **Si tu as Maths en spé complémentaire** : angle C est très différenciant.
- **Si tu vises une CPGE** ou une école d'ingénieur : angle B ou C.
- **Si tu veux jouer la sécurité** avec une excellente maîtrise : angle A.
- **Si tu débutes en Python** : angle A, mais maîtrise vraiment le code.

Autres idées de sujets Grand Oral en NSI

Pour situer le **choix** des échecs par rapport aux autres **sujets Grand Oral** possibles en **NSI**, voici un panorama des **idées de sujets** les plus solides :

Sujet	Forces principales	Limites	Difficulté
Sécurité informatique (RSA, chiffrement)	Actualité forte, liée à l'**internet** moderne	Niveau math élevé pour RSA, souvent traité	Moyenne
Cryptographie quantique	Sujet de pointe, impressionnant	Très exigeant, peu de candidats tiennent	Élevée
Réseaux sociaux et **algorithmes** de recommandation	Forte dimension de société	Peu de code à montrer, vite hors-programme	Moyenne
Systèmes embarqués (Raspberry Pi, IoT)	Très concret si projet matériel	Décrire le projet à l'oral ; préparer un schéma pendant les 20 minutes	Moyenne
Échecs et IA *(notre choix)*	Combine algo + structures + IA, imaginaire culturel	Risque du récit pur si mal préparé	Faible avec ce guide
Bases de données massives (Lichess, GitHub)	Cas concret de big data, SQL réel	Demande de manier statistiques	Moyenne
Compression d'images / sons (PNG, MP3)	Math + structure	Sujet technique exigeant	Moyenne
Tri et **complexité** algorithmique	Cœur du programme	Considéré scolaire, difficile de **révolutionner**	Faible
Réseaux de neurones génératifs (ChatGPT, LLM)	Actualité brûlante	Risque de l'exposé non technique	Élevée
Cryptographie monétaire (Bitcoin, blockchain)	Économie + **informatique**	Sujet glissant : faux experts en circulation	Moyenne

Pourquoi les échecs gagnent

Trois critères distinguent un bon **sujet** d'un **sujet** moyen :

1. **Il contient du code visible** que tu peux décrire à voix haute

2. **Il a un enjeu** identifiable au-delà de la technique (société, science, philosophie)
3. **Il s'attache à un domaine** que le **jury** reconnaît instantanément

Les échecs cochent les trois. La **cryptographie** coche les deux premiers mais peu le troisième pour un **jury** lettres. Les **réseaux sociaux** cochent les deux derniers mais pas le premier.

Construire ta problématique : méthode pas à pas

Étape 1, Choisir un angle parmi les trois

Tu choisis A, B ou C selon ton niveau. Marque ce **choix** sur ton brouillon en gros caractères, n'en change plus.

Étape 2, Tester la problématique avec le « test du tiers »

Énonce ta problématique à un tiers (parent, ami, professeur d'une autre discipline). S'il dit « je vois ce que tu vas raconter », c'est bon. S'il dit « explique-moi mieux », c'est trop technique ou trop flou : reformule.

Étape 3, Vérifier qu'elle est *problématisable*

Une bonne problématique présente une **tension** : « comment » plutôt que « qu'est-ce que ». Elle suggère qu'il y a un débat ou un mécanisme à comprendre. « Qu'est-ce que minimax ? » est une mauvaise problématique (descriptive). « Comment minimax permet-il de jouer en **temps** raisonnable ? » est meilleure (causale).

Étape 4, Vérifier qu'elle s'aligne avec ton orientation

Si tu vises une école d'ingénieur, ta problématique doit pouvoir conduire au passage « voilà pourquoi je veux étudier l'**informatique** ». Si elle ne le permet pas, c'est qu'elle est mal posée.

Script minuté : problématique B (optimisation alpha-bêta)

Les transitions rédigées sont en italique. Le code Python est dans des blocs à expliquer oralement : tu n'as pas à le lire mot à mot, mais à l'expliquer en français clair.

0:00-1:00, Introduction et problématique

> *« Bonjour. Je vais vous parler de la façon dont un ordinateur joue aux échecs, et plus précisément, comment il choisit son coup en un **temps** raisonnable.*
>
> *Shannon propose un ordre de grandeur historique de 10^120 pour l’arbre des parties, sous des hypothèses simplifiées. Ce n’est pas un dénombrement exact. Explorer naïvement toutes les suites est impraticable avec les moyens actuels. Ma problématique : en quoi l'élagage alpha-bêta illustre-t-il l'optimisation d'une recherche arborescente ?*
>
> *Je développerai en trois étapes : l'algorithme minimax de base, l'élagage alpha-bêta qui le rend efficace, puis les **limites** de cette approche et ce qu'AlphaZero a changé. »*

1:00-4:00, Partie 1 : l'algorithme minimax

> *« L'idée de base : modéliser le jeu comme un arbre. Chaque nœud est une position, chaque branche un coup possible. Le joueur blanc cherche à maximiser son avantage, le noir à le minimiser : d'où le nom minimax.*
>
> *Voici le code en Python. »*

Le code suivant est exécutable sur un petit arbre de scores. `Position` simule les transitions : pour jouer aux échecs, il faut la remplacer par une représentation du plateau et un générateur de coups légaux. Les scores restent toujours exprimés du point de vue des Blancs.

```
class Position:
    def __init__(self, arbre):
        self.arbre = arbre
        self.historique = []

    def est_terminee(self):
        return isinstance(self.arbre, (int, float))

    def coups_possibles(self):
        return range(len(self.arbre))

    def joue(self, coup):
        self.historique.append(self.arbre)
        self.arbre = self.arbre[coup]

    def annule(self, coup):
        self.arbre = self.historique.pop()


def evaluer(position):
    # À profondeur limitée, 0 est ici une évaluation illustrative.
    return position.arbre if position.est_terminee() else 0


def minimax(position, profondeur, maximise):
    # Cas de base : profondeur atteinte ou partie terminée
    if profondeur == 0 or position.est_terminee():
        return evaluer(position)   # retourne un score numérique

    if maximise:  # tour des Blancs : on cherche le maximum
        meilleur = float("-inf")
        for coup in position.coups_possibles():
            position.joue(coup)
            score = minimax(position, profondeur-1, False)
            position.annule(coup)
            meilleur = max(meilleur, score)
        return meilleur

    else:  # tour des Noirs : on cherche le minimum
        meilleur = float("inf")
        for coup in position.coups_possibles():
            position.joue(coup)
            score = minimax(position, profondeur-1, True)
            position.annule(coup)
            meilleur = min(meilleur, score)
        return meilleur
```

> *« Trois points clés : la récursivité (la fonction s'appelle elle-même), l'alternance maximise/minimise (les deux joueurs alternent), et la fonction `evaluer()` qui attribue un score à chaque position.*
>
> *Quel coût ? Dans un arbre uniforme hypothétique avec b = 35 et d = 4, il y a 35^4 = 1 500 625 feuilles ; avec d = 6, 35^6 = 1 838 265 625 feuilles. Les nœuds internes s'ajoutent. La notation O(b^d) décrit la croissance asymptotique du coût, pas un nombre exact d'appels. »*

4:00-7:00, Partie 2 : l'élagage alpha-bêta

> *« Le problème du minimax : il explore des branches inutiles. L'élagage alpha-bêta coupe ces branches sans changer le résultat.*
>
> *L'idée : si je sais déjà qu'une branche ne peut pas améliorer ma meilleure option connue, je l'ignore. »*

```
def alpha_beta(position, profondeur, alpha, beta, maximise):
    if profondeur == 0 or position.est_terminee():
        return evaluer(position)

    if maximise:
        meilleur = float("-inf")
        for coup in position.coups_possibles():
            position.joue(coup)
            score = alpha_beta(position, profondeur-1, alpha, beta, False)
            position.annule(coup)
            meilleur = max(meilleur, score)
            alpha = max(alpha, meilleur)
            if beta <= alpha:     # ← coupure bêta
                break             # cette branche ne sera jamais choisie
        return meilleur

    else:
        meilleur = float("inf")
        for coup in position.coups_possibles():
            position.joue(coup)
            score = alpha_beta(position, profondeur-1, alpha, beta, True)
            position.annule(coup)
            meilleur = min(meilleur, score)
            beta = min(beta, meilleur)
            if beta <= alpha:     # ← coupure alpha
                break             # cette branche ne sera jamais choisie
        return meilleur
```

> *« `alpha` est une borne inférieure disponible pour MAX ; `beta` une borne supérieure disponible pour MIN, dans la même échelle de scores. Ici MAX correspond aux Blancs. Quand `beta ≤ alpha`, une option déjà trouvée ailleurs rend inutile la suite de cette branche pour le choix d'un ancêtre. »*

> *Gain pratique : dans le meilleur cas (coups triés par ordre de qualité), alpha-bêta réduit le coût à O(b^(d/2)). Pour d = 4, l'ordre de grandeur passe de 35^4 à 35^2. Ce sont des ordres de grandeur asymptotiques, pas un comptage exact des feuilles ni une garantie de durée. Le gain dépend de l'ordre des coups. »*

7:00-9:00, Partie 3 : limites et rupture AlphaZero

> *« L'élagage alpha-bêta est élégant, mais avec des **limites** structurelles.*
>
> ***Limite 1 : la fonction d'évaluation.*** `evaluer()` *est écrite par des humains. Elle encode les intuitions d'experts (valeur des pièces, contrôle du centre, sécurité du roi). Ces règles peuvent être incorrectes ou incomplètes.*
>
> ***Limite 2 : la profondeur.*** *La profondeur accessible dépend de la position, du temps, du matériel et de la sélection des variantes. Une profondeur annoncée par un moteur ne signifie pas que toutes les branches ont été développées uniformément aussi loin.*
>
> ***La rupture d'AlphaZero (2017) :*** *AlphaZero n'a pas de fonction d'évaluation écrite par des humains. Il apprend à jouer en jouant contre lui-même (apprentissage par renforcement) et développe sa propre évaluation à travers un réseau de neurones. Le préprint de 2017 compare AlphaZero à Stockfish 8 à une minute par coup ; les quatre heures sont un repère sur la courbe d'apprentissage, pas la durée totale de tous les entraînements.*
>
> *AlphaZero apprend une politique et une évaluation qui guident MCTS. Il continue donc à explorer un arbre ; l'apprentissage change la sélection et l'évaluation, sans supprimer la recherche. »*

9:00-10:00, Conclusion

Pour les autres angles : variantes de script

Si tu choisis l'**angle A (arbre de décision)** ou l'**angle C (rupture AlphaZero)**, voici les bascules à effectuer :

Variante angle A (arbre de décision)

- Remplace la partie 2 (alpha-bêta) par une **partie 2 dédiée à la fonction d'évaluation** : comment on attribue un score numérique à une position. Évoque les pondérations classiques : dame = 9, tour = 5, fou/cavalier = 3, pion = 1, plus bonus de position.
- Garde minimax en partie 1 mais simplifie le code (sans alpha-bêta).
- En partie 3, parle de Deep Blue (1997) plutôt qu'AlphaZero.

Variante angle C (rupture AlphaZero)

- Inverse la balance : 2 minutes sur minimax/alpha-bêta (rapidement), puis 5 minutes sur AlphaZero.
- Insiste sur les **systèmes** de réseaux résiduels, MCTS guidé par politique, apprentissage par renforcement (auto-jeu).

- Termine par une **ouverture** sur d'autres problèmes de décision : quelles connaissances, données et règles faudrait-il fournir ? Un succès aux échecs ne prouve pas un transfert automatique vers un autre domaine.

Anticiper l'épreuve : 25 questions de jury rédigées

*Le **jury NSI** pose souvent des **questions** sur le code, le coût, et les distinctions algorithmiques. Prépare-toi à tracer une exécution à la main sur brouillon.*

Niveau 1, Comprendre le vocabulaire (questions de base)

Q1. Pourquoi avoir choisi les échecs pour votre Grand Oral NSI ?

> *« Les échecs concentrent plusieurs notions clés du **programme** de spé : la récursivité avec minimax, les arbres de décision et le coût algorithmique. AlphaZero permet ensuite une ouverture vers l'apprentissage automatique. C'est un cas d'étude historiquement documenté (de Deep Blue en 1997 à AlphaZero en 2017) qui permet de voir l'évolution des paradigmes en intelligence artificielle. »*

Q2. Expliquez la récursivité dans minimax en une phrase.

> *« Minimax est récursif parce qu'il définit la valeur d'une position à partir des valeurs de ses positions enfants, et chaque position enfant est évaluée par le même minimax, jusqu'à atteindre une profondeur limite ou une position terminale. »*

Q3. Qu'est-ce qu'un arbre de jeu ?

> *« Un arbre de jeu est une structure de **données** où la racine représente la position actuelle, chaque nœud représente une position du jeu, et chaque arête représente un coup possible. Les feuilles sont soit des positions terminales (fin de partie), soit des nœuds où on atteint la profondeur limite et où la fonction d'évaluation s'applique. »*

Q4. Quelle est la différence entre un nœud MAX et un nœud MIN dans minimax ?

> *« Un nœud MAX correspond au tour du joueur qui veut maximiser son score : typiquement les Blancs. On y choisit le coup qui donne le score le plus élevé parmi les enfants. Un nœud MIN correspond au tour du joueur qui veut minimiser le score : les Noirs. L'alternance des nœuds MAX et MIN simule les deux joueurs qui prennent des décisions optimales à tour de rôle. »*

Q5. Illustrez le coût de minimax pour b=30 et d=4 niveaux.

> *« L'arbre uniforme possède 30^4 = 810 000 feuilles, sans compter les nœuds internes. Dans le meilleur cas, O(b^(d/2)) conduit à comparer cette puissance à 30^2 = 900. Leur rapport vaut 900 ; ce n'est ni le compte exact des feuilles visitées par alpha-bêta ni un gain mesuré de durée. »*

Niveau 2, Comprendre les mécanismes (questions intermédiaires)

Q6. Expliquez la condition `if beta <= alpha : break` dans le code alpha-bêta.

> *« `alpha` est une borne inférieure pour MAX, `beta` une borne supérieure pour MIN, toutes deux dans la même échelle de scores. Quand `beta ≤ alpha`, une option déjà trouvée ailleurs suffit à rendre inutile la suite de cette branche. La coupure peut se produire aussi bien à un nœud MAX qu'à un nœud MIN. »*

Q7. Quelle est la différence entre le coût dans le meilleur cas et dans le pire cas pour alpha-bêta ?

> *« Avec un ordre idéal, le coût usuel est O(b^(d/2)) ; dans le pire cas, il peut rester O(b^d). Cela ne signifie pas qu'on coupe la moitié des branches à chaque niveau. Pour un tri heuristique réel, le gain doit être mesuré sur les arbres, positions et budgets étudiés ; aucun exposant intermédiaire universel n'est garanti. »*

Q8. Qu'est-ce que la fonction d'évaluation et pourquoi est-elle cruciale ?

> *« La fonction d'évaluation attribue un score numérique à chaque position non terminale atteinte à la profondeur limite. Dans une version écrite à la main, elle peut combiner des critères comme le matériel, le contrôle du centre ou la sécurité du roi ; une autre version peut être apprise par un réseau. Ce score est une estimation, pas une preuve. Sa qualité et la profondeur de recherche influencent ensemble les décisions. »*

Q9. Pourquoi dit-on que l'algorithme minimax suppose un adversaire optimal ?

> *« Dans notre convention de scores, MAX choisit le maximum et son adversaire MIN le minimum. Sur un arbre entièrement résolu, cette stratégie garantit à MAX sa valeur minimax, même si MIN peut se tromper. Elle ne cherche pas à deviner les erreurs d'un adversaire particulier. À profondeur limitée, cette garantie porte seulement sur l'arbre et les évaluations utilisés, pas sur l'issue réelle de la partie. »*

Q10. Comment Deep Blue (1997) différait-il de l'algorithme minimax pur ? Deep Blue combine une recherche optimisée, une évaluation et des ressources spécialisées. IBM documente sa victoire contre Kasparov en 1997 ; une bibliothèque d'ouvertures n'impose pas un nombre universel de vingt coups joués sans recherche.

Q11. Qu'est-ce que l'apprentissage par renforcement qu'AlphaZero utilise ?

> *« L'apprentissage par renforcement est un paradigme où l'agent apprend en interagissant avec son environnement et en recevant des récompenses. AlphaZero joue contre lui-même (self-play) : il reçoit +1 s'il gagne, -1 s'il perd, 0 pour une nulle. Le réseau ajuste ses paramètres pour rapprocher sa valeur du résultat final et sa politique des probabilités produites par la recherche. Il apprend sans exemples de parties humaines ni liste de critères stratégiques fournie à la main. Les règles, l'architecture et la procédure d'entraînement restent définies par des humains. »*

Q12. Quelle est la structure du réseau de neurones dans AlphaZero ?

> *« AlphaZero utilise un réseau résiduel profond (ResNet) avec deux têtes de sortie. La tête de valeur retourne un nombre entre -1 et 1 : une estimation du résultat attendu depuis cette position, en tenant compte des nulles. La tête de politique retourne une distribution de probabilité sur les coups possibles : lesquels méritent d'être explorés. Ces deux sorties guident une recherche Monte Carlo Tree Search (MCTS) qui remplace le minimax classique. »*

Niveau 3, Nuances et limites (questions avancées)

Q13. Pourquoi AlphaZero est-il plus difficile à analyser qu'un programme minimax ?

> *« On peut suivre la remontée des scores et les coupures de notre petit programme. Mais une évaluation, même écrite à la main, demande encore une justification. Avec un réseau appris, relier les paramètres à une explication stratégique simple est plus difficile ; on peut toutefois examiner ses sorties et sa recherche. Il faut distinguer trace de calcul, interprétation et preuve, sans conclure qu'aucune analyse du réseau n'est possible. »*

Q14. Peut-on appliquer l'algorithme minimax à d'autres jeux que les échecs ?

> *« Minimax s'applique aux jeux finis à deux joueurs, à information parfaite et à somme nulle. Le morpion en est un exemple simple. Puissance 4 a été résolu en 1988 : le premier joueur dispose d'une stratégie gagnante sur la grille standard. Le mémoire d'Allis utilise notamment des règles stratégiques et une recherche adaptée. Gagner au Go grâce à AlphaGo ne signifie pas avoir résolu le Go. »*

Q15. Comment le tri des coups améliore-t-il alpha-bêta en pratique ?

> *« Le tri cherche à examiner d'abord les coups les plus prometteurs pour le joueur au trait. À MAX, une valeur élevée peut relever* `alpha` *; à MIN, une valeur basse peut abaisser* `beta` *. Des bornes resserrées tôt peuvent permettre davantage de coupures dans la suite de la recherche. Les coups conservés dans une table de transposition et les heuristiques d'historique peuvent guider ce tri. Ils ne garantissent ni l'ordre idéal ni le meilleur cas O(b^(d/2)) : leur gain dépend des positions et doit être mesuré. »*

Q16. Qu'est-ce qu'une table de transposition en algorithmique des jeux ?

> *« Une table de transposition est une table de hachage qui mémorise les positions déjà évaluées avec leur score. Aux échecs, la même position peut être atteinte par des séquences de coups différentes ; sans table de transposition, on l'évalue plusieurs fois. Une entrée peut aussi préciser la profondeur explorée, un coup et la nature du score : valeur exacte pour cette recherche, borne inférieure ou borne supérieure. On vérifie ces conditions avant de réutiliser le résultat ; retrouver une position ne suffit pas à retourner n'importe quel ancien score. C'est une forme de mémoïsation adaptée à la recherche arborescente. »*

Q17. Pourquoi les bitboards accélèrent-ils les programmes d'échecs ?

> *« Un bitboard représente l'échiquier comme un entier de 64 bits, où chaque bit correspond à une case. Un masque peut représenter, par exemple, les cases occupées par les pions blancs. AND, OR et les décalages permettent de combiner ces ensembles de cases avec peu d'opérations. Cela ne vérifie pas tous les coups légaux en une instruction : les règles, les bords du plateau et les échecs au roi demandent d'autres calculs. Le gain dépend du matériel et de l'implémentation ; on le mesure plutôt que d'annoncer un facteur universel. »*

Q18. Stockfish est-il toujours de type alpha-bêta ou a-t-il adopté des réseaux de neurones ?

> *« Depuis Stockfish 12 (2020), Stockfish intègre NNUE (Efficiently Updatable Neural Network). C'est un réseau de neurones utilisé comme fonction d'évaluation, mais l'algorithme de recherche reste alpha-bêta. C'est une approche hybride : la structure de recherche classique avec une évaluation apprise par réseau de neurones. L'annonce officielle de Stockfish 12 documente cette intégration. Un gain Elo exige un protocole de test précis ; l'adoption de NNUE ne transforme pas le moteur en jeu résolu et ne permet pas de prédire le classement actuel des moteurs. »*

Q19. Comment AlphaZero utilise-t-il Monte Carlo Tree Search ?

> *« AlphaZero combine le réseau de neurones avec MCTS. Le réseau guide quels nœuds explorer en priorité (la tête de politique donne des probabilités sur les coups). La recherche accumule des visites et des estimations de valeur, en combinant ce qu'elle a déjà observé avec les priorités de la politique. Le choix final dépend du protocole et du réglage de température ; ce n'est pas simplement tirer des parties au hasard. Le réseau est entraîné sur les résultats de ces recherches. C'est une boucle : le réseau guide MCTS, et MCTS produit des* ***données*** *pour améliorer le réseau. »*

Q20. Un algorithme parfait aux échecs est-il possible en principe ?

> *« Un jeu fini comme les échecs possède une valeur sous jeu parfait. Minimax pourrait la calculer en explorant l'arbre jusqu'aux positions terminales ; la profondeur serait finie, mais démesurée. Les estimations combinatoires expliquent pourquoi la force brute est impraticable aujourd'hui. Elles ne démontrent pas que toute résolution future est impossible. »*

Niveau 4, Ouvertures et liens transversaux

Q21. Les techniques d'IA des échecs s'appliquent-elles à d'autres domaines ?

> *« Des idées de recherche et d'apprentissage peuvent être réutilisées, mais chaque domaine demande de définir ses données, actions, objectifs et critères d'évaluation. La réussite d'AlphaZero aux échecs ne démontre pas qu'un système médical ou un autre modèle en dérive directement, ni qu'il aura la même performance. Pour ton ouverture, explique une méthode transférable plutôt qu'une filiation non établie. »*

Q22. Quels enjeux de sécurité informatique soulève l'IA dans les échecs en ligne ?

> *« L'aide extérieure non autorisée pose un problème de triche. Il faut lire le règlement applicable : Lichess interdit les moteurs dans les parties humaines en temps réel et distingue d'autres usages, comme les parties contre son IA ou l'API des bots. Pour parler de détection, distingue un indice statistique d'une preuve : jouer plusieurs coups recommandés par un moteur ne suffit pas, isolément, à démontrer une assistance. C'est un cas de sécurité où les données, les faux positifs et la procédure de décision comptent. »*

Q23. Quel rôle jouent les bases de données dans les programmes d'échecs ? Distingue les bibliothèques d'ouvertures, les bases de parties PGN et les tablebases de finales. La couverture des tablebases dépend notamment des droits de roque et des métriques ; un nombre de pièces ne supprime pas toutes les exceptions. Lichess publie des parties en téléchargement : date et périmètre doivent accompagner un éventuel décompte.

Q24. Comment les échecs illustrent-ils le débat sur les limites de l'IA ?

> *« Deep Blue en 1997 et AlphaZero en 2017 illustrent deux démarches techniques et des performances dans des protocoles déterminés. AlphaZero apprend par autojeu dans un cadre construit par des humains. Un résultat de match n'établit ni une intelligence générale ni une stratégie parfaite. Pour comparer un modèle de langage, il faudrait préciser sa version et un test reproductible, pas lui attribuer un niveau par anecdote. »*

Q25. Si tu devais reprendre ton sujet dans dix ans, qu'y changerais-tu ?

> *« En dix ans, l'IA aura probablement intégré encore plus le quotidien. Je m'attendrais à ce que les programmes d'échecs soient utilisés pour comprendre les **réseaux** de neurones eux-mêmes : pourquoi un coup est-il jugé meilleur qu'un autre ? Les chercheurs commencent à « interpréter » AlphaZero, à extraire des principes stratégiques que le réseau a découverts. Dans dix ans, ce sera peut-être la principale leçon : utiliser les IA pour comprendre les IA. »*

Anti-sèche imprimable : à plier dans ta poche

Voici une fiche mémo concentrée que tu peux imprimer ou recopier sur une feuille A5. Quand tu doutes, tu sors cette fiche dans ta tête.

Les trois chiffres à connaître par cœur

- **35** : branchement uniforme choisi pour les exemples (b), pas le nombre de coups légaux dans toute position
- **10^120** : ordre de grandeur historique de Shannon sous ses hypothèses, pas dénombrement exact
- **28-0-72** : score d'AlphaZero vs Stockfish en 2017 (100 parties)

Les trois dates pivots

- **1950** : Shannon publie son article sur la programmation des échecs
- **1997** : Deep Blue bat Kasparov (symbolique, force brute)

- **2017** : préprint AlphaZero ; quatre heures comme repère de sa courbe d'évaluation, pas durée universelle d'apprentissage

Les trois notations de coût à manier

- **O(b^d)** : minimax pur (exponentielle)
- **O(b^(d/2))** : alpha-bêta meilleur cas (racine)
- **O(1)** : coût moyen usuel d'accès à une table de hachage sous des hypothèses ; pas une garantie universelle

Les trois mots-clés à placer

- **Récursivité** (cœur de minimax)
- **Mémoïsation** (table de transposition)
- **Renforcement** (apprentissage AlphaZero)

Fiche mémo NSI complète

```
╔═══════════════════════════════════════════════════════════╗
║          COÛTS ALGORITHMIQUES, GRAND ORAL NSI             ║
╠═══════════════════════════════════════════════════════════╣
║ MINIMAX (sans optimisation)                               ║
║   Coût temporel : O(b^d)                                  ║
║   b = branchement choisi pour l'exemple (35)              ║
║   d = profondeur de recherche                             ║
║   Arbre uniforme : 35^4 = 1 500 625 feuilles              ║
╠═══════════════════════════════════════════════════════════╣
║ ALPHA-BÊTA (meilleur cas : coups bien triés)              ║
║   Coût temporel : O(b^(d/2))                              ║
║   Puissance indicative : 35^2 = 1 225 (pas un compte exact) ║
║   Rapports des puissances : 1 225 et 42 875, non gains réels║
╠═══════════════════════════════════════════════════════════╣
║ PARAMÈTRES ALPHA-BÊTA                                     ║
║   alpha : borne inférieure disponible pour MAX            ║
║   beta  : borne supérieure disponible pour MIN            ║
║   Coupure : si beta ≤ alpha → branche ignorée             ║
╠═══════════════════════════════════════════════════════════╣
║ VOCABULAIRE CLÉ                                           ║
║   Récursivité   : fonction qui s'appelle elle-même        ║
║   Élagage       : suppression de branches inutiles        ║
║   Heuristique   : règle approchée pour guider la recherche ║
║   Mémoïsation   : stocker pour éviter recalcul            ║
║   Transposition : même position via chemins différents    ║
╠═══════════════════════════════════════════════════════════╣
║ COMPARAISON DES APPROCHES                                 ║
║   Minimax / Alpha-bêta : règles explicites, explicable    ║
║   AlphaZero / MCTS+NN  : évaluation et politique apprises ║
║   Stockfish NNUE       : hybride (alpha-bêta + réseau)    ║
╚═══════════════════════════════════════════════════════════╝
```

Conseils pour réussir le jour J

Posture et présence

Tu es debout pendant les **dix minutes** d'exposé. Les détails comptent :

- **Pieds ancrés** à largeur d'épaules, **jamais** croisés
- **Mains visibles** : sur la table, en mouvement, jamais dans les poches
- **Regard distribué** entre les deux membres du **jury**, sans quota chiffré à respecter
- **Voix** : projetée, posée, **pas** monotone. Marque les pauses après les chiffres clés
- **Sourire** au moins une fois pendant l'introduction et une fois en conclusion

Gestion du stress avant l'épreuve

Le **stress** d'avant **oral** est fréquent ; son effet varie selon les personnes et son intensité. Voici trois exercices à essayer pendant les répétitions, sans résultat garanti :

1. **Respiration carrée** (4-4-4-4) : inspire 4 s, retiens 4 s, expire 4 s, pause 4 s. Trois cycles juste avant d'entrer dans la salle.
2. **Ancrage corporel** : pieds bien à plat dans le sol, poings serrés 5 secondes puis relâche. Reconnecte au corps.
3. **Premier mot automatique** : ta première phrase doit être prête par cœur. Cela peut t'aider à démarrer sans chercher tes mots.

Gestion du temps pendant l'exposé

- **Chronométrage discret** : pose ta montre face à toi, ou utilise un chrono visible. Si tu n'as ni l'un ni l'autre, marque mentalement à 5 minutes (mi-exposé).
- **Plan en repère** : à chaque transition, dis « première partie », « deuxième partie » : cela aide le **jury** à suivre **et** te recale dans ta progression.
- **Si tu accélères** par stress : repère un fait technique (un chiffre, un nom) et ralentis dessus. Une seconde de pause vaut mieux qu'un débit qui s'emballe.

Pendant l'échange : 10 minutes de questions

Le **jury** te questionne. Quelques règles :

- **Reformule la question** dans tes mots avant de répondre. Cela te donne du temps de réflexion.
- **Si tu ne sais pas**, dis « je ne suis pas certain mais je dirais que… » plutôt que « je ne sais pas » sec.
- **Distingue les niveaux** : si on te demande « définissez la récursivité », commence par une phrase simple puis nuance.
- **Quand tu ne comprends pas**, demande poliment « pourriez-vous reformuler ? » plutôt que de répondre à côté.

Ton projet d'orientation pendant l'échange

Depuis la session 2024, l’orientation n’est plus une troisième partie chronométrée. L’échange approfondit la présentation ; il peut conduire à expliquer ton choix de sujet, sans promettre qu’une question d’orientation sera posée. Prépare une réponse courte et cohérente :

- Si tu vises une **CPGE** : « Ce **sujet** m'a fait découvrir le raisonnement formel, c'est ce que je veux approfondir en maths-info ou MPI. »
- Si tu vises une **école d'ingénieur** post-bac : « L'algorithmique appliquée à un cas concret m'a passionné, je veux continuer dans un cursus pratique comme l'INSA ou l'UTC. »
- Si tu vises un **BUT informatique** : « J'ai préféré le code à la théorie pure, je veux un cursus qui privilégie la pratique. »
- Si tu vises une **licence informatique** ou **mathématiques** : « J'ai aimé manipuler les **mathématiques** discrètes et les structures de **données**, je veux les creuser en université. »

Important : ne mens pas sur ton projet. Le **jury** sent l'incohérence. Un projet « en construction » est mieux accepté qu'un projet inventé.

Check-list de préparation

J-30 (un mois avant)

- [ ] Choisir la problématique parmi les trois proposées
- [ ] Implémenter minimax simple en Python (même pour morpion, c'est suffisant)
- [ ] Tracer l'arbre minimax à la main pour une position à 2 niveaux (exercice essentiel)
- [ ] Mémoriser les trois dates : 1950, 1997, 2017
- [ ] Lire au moins deux **questions** du **jury** par jour à voix haute

J-15 (deux semaines avant)

- [ ] Premier entraînement chrono : 10 minutes exposé complet, seul, devant miroir
- [ ] Lister les **mathématiques** spécifiques à maîtriser (b^d, racine carrée, logarithme)
- [ ] Utiliser les feuilles de code imprimées pour réviser ; s'entraîner à préparer un court support sur papier pendant les 20 minutes
- [ ] Identifier deux ou trois **exemples** numériques à connaître par cœur (35^4, 30^3, etc.)

J-7 (une semaine avant)

- [ ] Deuxième entraînement chrono avec un proche jouant le **jury**
- [ ] Répondre à au moins 10 **questions** du **jury** à voix haute
- [ ] Être capable d'expliquer chaque ligne du code sans le lire
- [ ] Revoir la fiche mémo complète

J-2 (l'avant-veille)

- [ ] Dernier entraînement complet, chrono pris, exposé filmé pour révision
- [ ] Vérifier la tenue (pantalon ou jupe propre, chemise/chemisier, chaussures fermées)
- [ ] Préparer le sac : pièce d'identité, convocation, stylos, eau

J-1 (la veille)

- [ ] Relire l'anti-sèche imprimable (cinq minutes max, **pas** plus)
- [ ] Vérifier l'heure et l'adresse de la convocation
- [ ] Se coucher tôt : le **stress** dérègle déjà le sommeil, anticipe

Jour J

- [ ] Manger normalement le matin (**pas** plus, **pas** moins ; ventre vide accentue le stress)
- [ ] Arriver 30 minutes en avance
- [ ] Eau dans la salle d'attente, **pas** de café à jeun
- [ ] Phase de préparation : 20 minutes pour organiser la réponse à la question choisie par le jury.

Deux fiches intégrées : minimax et alpha-bêta

Les développements des sujets minimax et alpha-bêta sont regroupés ici. Les fiches minimax et alpha-bêta restent disponibles pour une révision courte.

Remonter les scores sur un exemple autonome

Avec deux nœuds MIN sous une racine MAX, prends les feuilles (3, 5) puis (2, 9). MIN retient 3 dans la première branche et 2 dans la seconde ; MAX choisit donc 3. Le code complet présenté plus haut produit ce résultat sur `Position(((3, 5), (2, 9)))`.

Dans la version alpha-bêta, après avoir obtenu 3 sur la première branche, MAX n'a plus besoin d'explorer la feuille 9 : MIN a déjà rencontré 2 dans la seconde, qui ne pourra donc pas devenir préférable au 3 disponible ailleurs. C'est une coupure justifiée par les choix des deux joueurs, pas par l'idée qu'une feuille serait intrinsèquement sans intérêt.

Présenter la complexité avec ses conditions

Pour b = 35 et d = 6, le rapport des puissances b^d et b^(d/2) vaut 35^3 = 42 875. C'est une comparaison d'ordres de grandeur dans le meilleur cas de tri, pas un gain mesuré ou un décompte exact. Sans bon ordre des coups, alpha-bêta peut retrouver le coût de minimax.

Une feuille évaluée à profondeur limitée n'est pas nécessairement une fin de partie. Distingue donc la valeur exacte d'un arbre entièrement résolu et le score estimé par une fonction heuristique. Le code de démonstration et son évaluation sont définis ; un moteur jouable demanderait en plus les règles et la génération des coups.

Sources et références

- Ministère de l'Éducation nationale (2026), *Baccalauréat : comment se passe le Grand oral ?* Présentation officielle.
- Russell, S. et Norvig, P. (s. d.), *Adversarial Search*, projet de chapitre d'Artificial Intelligence: A Modern Approach, Berkeley. Texte.
- Allis, V. (1988), *A Knowledge-based Approach of Connect-Four*, Vrije Universiteit Amsterdam. Mémoire.
- Shannon, C. E. (1950), *Programming a Computer for Playing Chess*, Philosophical Magazine, 41, 256-275. Article.
- Silver, D. et al. (2017), *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, préprint arXiv. Article.
- IBM (s. d.), *Deep Blue*, histoire du projet. Présentation.
- Équipe Stockfish (2020), *Stockfish 12*. Annonce de NNUE.
- Équipe Stockfish (s. d.), code source des entrées de table de transposition et des opérations sur les bitboards.
- Lichess (s. d.), *Fair Play*, règles selon le mode de jeu. Règlement.
- Projet Syzygy (s. d.), *probetool*, documentation des tablebases. Documentation.
- Lichess (s. d.), *Lichess Open Database*. Données.

*Ce guide est librement utilisable et imprimable. Si d'autres lycéens préparent le même **sujet**, partage-leur le lien. Et bonne chance : tu as fait le travail.*

Pour aller plus loin

- Sujet Grand Oral NSI : En quoi l'élagage alpha-bêta illustre-t-il l'optimisation d'une recherche arborescente ?
- Sujet Grand Oral NSI : Comment un programme informatique peut-il jouer aux échecs ?
- Sujet Grand Oral NSI : Pourquoi AlphaZero représente-t-il une rupture dans l'intelligence artificielle ?